

Edge cases (search comes back empty) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, plus how a real RAG system handles the messy edge cases). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Ask a question that's clearly outside your document set's coverage (like asking a policy document about the weather). Your search will come back with nothing genuinely relevant.

The question this exercise really tests: what does your graph do with "nothing found"? Does it notice and say so honestly, or does it hand the model an empty (or near-empty) block of "context" and let it guess an answer anyway?

Things to use:

- Your search node, saving its results into the state (an empty list if nothing found).
- A new conditional edge, checked right after search, that looks at whether that list is empty.
- A small "can't answer" node that returns a clear, honest message instead of an answer.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This exercise is about adding an explicit check, not a new capability. After your search node runs, check whether it actually found anything worth using — and if not, route to a distinct "can't answer" response instead of letting the reasoning node try anyway with nothing to work from.

```
# empty_search_practice.py
def has_results(state: dict) -> str:
    if not state.get("sources"):
        return "no_grounding"
    return "reason"
```

The important design point: this check has to happen *after* search runs, as its own routing decision — not folded silently into the reasoning node's prompt, where "I found nothing" and "I found something thin" can blur together into a guess.

The exact pieces:

- **What "nothing useful" means** — an empty list is the obvious case.
- **The new node** — `cant_answer_node` should return something the caller can clearly tell apart from a real, grounded answer — a distinct field, or a clearly worded message, not just a short version of a normal answer.
- **Where this check lives** — a conditional edge right after the search node, checked before the reasoning node ever runs, so the model is never even given the chance to improvise from nothing.

An empty list is the easy case to catch. A harder, more common case: search returns *something*, but it's barely related — a vector similarity search almost never returns literally zero results, it returns the *closest* chunks it has, even when none of them are actually relevant. A thin, low-scoring result is just as misleading as no result at all, because your "nothing useful" check won't catch it — the list isn't empty, so the graph sails on to `reason_node` and the model dutifully "answers" from context that doesn't really support an answer.

The real design question isn't "is the list empty" — it's "how relevant does a result actually need to be before it counts as grounding, and what do you do with a result that's borderline?"

Answering that needs a similarity score, which most vector retrievers can give you alongside each chunk, and a threshold you choose deliberately (not an arbitrary guess — start from a few real examples of "this is genuinely relevant" versus "this is a coincidental near-miss," and pick a number that separates them).

The extra pieces needed:

- Your retriever returning a `(chunk, score)` pair per result instead of just the chunk, so the routing check has something to compare against a threshold.
- A `RELEVANCE_THRESHOLD` constant, and a check like `max(score for _, score in results) >= RELEVANCE_THRESHOLD` instead of just `not results`.
- A place to log every "no grounding" and "below threshold" event with the original task text — not to change behavior in this exercise, but because reviewing those logs later is how a real project discovers gaps in its document set (the same questions keep failing to ground → that content is missing, not just phrased badly).

Sketch the threshold check yourself before checking Hint 2.

Difference between Basic and Intermediate: Basic treats "found nothing" as "the list is empty" — true, but only half the problem. Intermediate adds the harder, more common case: a non-empty list that's still not good enough to ground an answer, caught with a relevance-score threshold instead of a length check, plus a logging habit that turns "search fails silently on some questions" into "here's the list of questions search actually fails on, and why" — the difference between noticing a gap once and being able to systematically close it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```

after search_node runs, state has "sources" (a list, maybe empty)

function has_results(state):
    if sources list is empty:
        return "no_grounding"
    return "reason"

add a new node cant_answer_node:
    return a message like "I don't have grounding for this in my documents"

wire:
    add_conditional_edges(after search_node, has_results,
        {"no_grounding": cant_answer_node, "reason": reason_node})

test:
    ask something outside your document set -> should land on cant_answer_node

```

Here's almost the whole thing — just try running it and reading it line by line:

```

# empty_search_practice.py
def has_results(state):
    if not state.get("sources"):
        return "no_grounding"
    return "reason"

def cant_answer_node(state):
    return {
        "answer": "I don't have grounding for this in my documents.",
        "grounded": False,
    }

graph.add_node("cant_answer_node", cant_answer_node)
graph.add_conditional_edges(
    "search_node", has_results,

```

```
{ "no_grounding": "cant_answer_node", "reason": "reason_node"},  
)
```

Expected output if you run just this: nothing on its own — invoke the graph with a fully out-of-scope question (like "what's the weather today?") and check that `result["answer"]` is the honest fallback message, not a guess.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
define:
    def has_results(state: dict) -> str:
        if not state.get("sources"):
            return "no_grounding"
        return "reason"

    def cant_answer_node(state: dict) -> dict:
        return {
            "answer": "I don't have grounding for this in my documents.",
            "grounded": False,
        }

register:
    graph.add_conditional_edges(
        "search_node",
        has_results,
        {"no_grounding": "cant_answer_node", "reason": "reason_node"},
    )

test:
    invoke with a question with zero overlap with your document set
    confirm the final state's "answer" is the honest fallback, not a guess
```

```
# empty_search_practice.py
def has_results(state: dict) -> str:
    sources = state.get("sources", [])
    if not sources:
        return "no_grounding"
    return "reason"

def cant_answer_node(state: dict) -> dict:
    return {
        "answer": "I don't have grounding for this in my documents.",
        "grounded": False,
    }
```

Register `cant_answer_node` with `graph.add_node(...)`, wire the conditional edge, and test both an in-scope and out-of-scope question before comparing against the [Solution](#).

Once that's working, replace "is it empty" with "is the best result actually good enough":

```
RELEVANCE_THRESHOLD = some number you chose from real examples

function has_results(state):
    scored_sources = state's sources, each with a similarity score
    if scored_sources is empty OR the best score is below RELEVANCE_THRESHOLD:
        log this task as "no grounding"
        return "no_grounding"
    return "reason"
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# empty_search_practice.py
import logging

logger = logging.getLogger("grounding")
RELEVANCE_THRESHOLD = 0.75

def has_results(state: dict) -> str:
    scored_sources = state.get("scored_sources", [])

    # your turn: compute the best (highest) score among scored_sources,
    # treating an empty list as "no score at all" -- then return
    # "no_grounding" if scored_sources is empty OR the best score is
    # below RELEVANCE_THRESHOLD, otherwise return "reason". Log the
    # task text either way you go, using logger.info(...).
    ...
```

Fill in the threshold check and the logging call yourself, then compare all of your finished versions against the [Solution](#).

Difference between Basic and Intermediate: Basic checks only whether the list is empty — a real search backend that always returns *some* chunks (just possibly bad ones) will sail right past that check. Intermediate replaces "is it empty" with "is the best result actually good enough," using a similarity-score threshold chosen from real examples, and adds a logging habit that turns silent failures into a reviewable record of exactly which questions your document set can't currently answer.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)